

Язык описания аппаратуры Verilog-HDL

Оглавление

1 Введение.....	1
1.1 Языки программирования и языки описания аппаратуры.....	2
1.2 Языки программирования	2
1.3 Язык описания аппаратуры.....	3
2 Иерархическое описание устройства.....	5
3 Модули (Modules).....	6
3.1 Порты.....	9
3.2 Параметры (Parameters)	10
4 Основные конструкции языка.....	11
4.1 Соглашения о именовании, комментарии	11
4.2 Числа.....	12
4.3 Строки.....	14
4.4 Типы данных.....	15
Цепи и регистры.....	15
Памяти.....	17
Целые числа и время.....	17
4.5 Операторы.....	18
Операнды (Operands)	18
Побитовые операторы (Bit-Wise Operators)	21
Операторы приведения.....	22
Операторы сдвига	24
Условный оператор	24
Конкатенация - Concatenations	25
Репликация.....	26
5 Назначения (Assignments)	26
5.1 Краткий обзор Назначений (присваиваний).....	26
5.2 Непрерывные присваивания.....	27
5.3 Назначение, выполняемое при объявлении цепи	27
5.4 Оператор непрерывного присваивания - assign.....	28
Сила сигнала.....	29
5.5 Задержки Delays.....	31
5.6 Минимальная, типовая, максимальная задержка (Minimum, Typical, Maximum Delay)	33
5.7 Процедурное назначение.....	34
Оператор Initial	34
Оператор always.....	35
Блокирующие и неблокирующие присваивания.....	38
6 Операторы ветвления.....	39
6.1 Утверждения if...else.....	39
6.2 Утверждения case.....	42
6.3 Оператор casex, casez.....	43
7 Иерархическое описание проекта.....	44
7.1 Подключение массива модулей.....	47

1 Введение

Язык Verilog был разработан фирмой Gateway Design Automaton как внутренний язык симуляции. Cadence приобрела Gateway в 1989 г. и открыла Verilog для общественного использования. В 1995 г. был определен стандарт языка — Verilog LRM (Language Reference Manual), IEEE1364-1995. Таким образом, датой появления языка Verilog следует считать 1995 г. К этому времени уже успел получить широкое распространение другой язык высокого уровня для описания принципиальных схем — VHDL (Very high-speed IC Hardware

Description Language), появившийся еще в 1987 г. Несмотря на похожие названия, Verilog HDL и VHDL — различные языки. Verilog — достаточно простой язык, сходный с языком программирования С как по синтаксису, так и по «идеологии». Малое количество служебных слов и простота основных конструкций упрощают изучение и позволяют использовать Verilog в целях обучения. Но в то же время это эффективный и специализированный язык. VHDL обладает большей универсальностью и может быть использован не только для описания моделей цифровых электронных схем, но и для других моделей (например, модели экосистемы). Однако из-за своих расширенных возможностей VHDL проигрывает в эффективности и простоте, то есть на описание одной и той же конструкции в Verilog потребуется в 3–4 раза меньше символов (ASCII), чем в VHDL.

В Verilog существуют специфические объекты (UDP, Specify-блоки), не имеющие аналогов в VHDL. Также следует упомянуть стандарт PLI (Program Language Interface), который позволяет включать функции, написанные пользователем (например, на С), в код симулятора. В настоящее время важным недостатком Verilog является отсутствие документации, особенно на русском языке.

Необходимо отметить, что язык Verilog первоначально разрабатывался исключительно для решения задач моделирования (verify logic), а VHDL — вообще для документирования цифровых схем путем их формального описания. Поэтому не все конструкции HDL могут использоваться для описания проекта на ПЛИС или, иначе, для синтеза устройства. Таким образом, в каждом языке описания аппаратуры можно выделить «синтезируемое подмножество», т.е. множество конструкций языка, которые могут быть использованы при синтезе цифровых схем средствами стандартных САПР.

1.1 Языки программирования и языки описания аппаратуры

В чем же разница и в чем сходство таких языков? Известно, что любой предмет можно и нужно описывать следующим образом:

1. в какой предметной области данный предмет применяется
2. цели — задачи, для которых он предназначен
3. из чего состоит предмет
4. как эти части взаимодействуют
5. как предмет взаимодействует с внешней средой.

1.2 Языки программирования

Язык программирования задает последовательность действий в фиксированной среде. Представим, что мы программируем микроконтроллер. Сам микроконтроллер от наших действий не меняется. Меняется только порядок обработки данных, поступающих в микроконтроллер и выдаваемых из микроконтроллера. При этом, программист микроконтроллеров знает, что структура микроконтроллера и описание его команд приводятся в комплекте документации на микроконтроллер. Микроконтроллер под управлением потока команд обрабатывает данные последовательно. Да, конечно, существуют процессоры, которые за один такт «умеют» выполнять несколько команд, есть каналы DMA, которые пересылают данные на периферийные узлы микроконтроллера для дальнейшей автономной обработки, но все же в целом, обработка данных идет последовательно. Представим, что мы хотим произвести инкремент 100 переменных. Тогда в тексте программы это будет выглядеть так:

X0++;
X1++;
....
X99++;

В первом приближении, можно сказать так: если представить, что каждая команда выполняется за один такт, то такой фрагмент программы будет выполнен микроконтроллером за 100 тактов.

1.3 Язык описания аппаратуры

Язык описания аппаратуры, в противоположность языку программирования, *описывает связи в устройстве*. А функционирование устройства будет происходить в соответствии с теми внешними воздействиями, которые будут к нему приложены. То-есть разработчик аппаратуры, в соответствии с заданием для разработки, выбирает состав устройства и описывает этот состав и все связи устройства, пользуясь тем, что алгоритм работы устройства описан в комплекте документации, или, если говорить более точно, в техническом задании, входящем в этот комплект. Поскольку составные части устройства могут работать независимо друг от друга, то для примера с инкрементированием 100 переменных на аппаратном уровне нужно будет всего только 1 такт. В тексте описания аппаратуры, так же как и в тексте описания программы будут присутствовать 100 строк примерно такого вида:

$X_n = X_n + 1;$

Таким образом внешне все очень похоже. Но только внешне. А по сути это будут 100 одновременных процессов. Таким образом необходимо еще раз подчеркнуть, что все процессы, описанные на языке описания аппаратуры идут одновременно. Конечно, языки описания аппаратуры кроме возможностей предоставляемых структурным описанием имеют набор возможностей и для поведенческого описания. Да, можно описать то, как должно работать устройство. Но здесь необходимо отметить, что поведенческое описание не всегда можно превратить в загружаемый в микросхему FPGA файл. Только самые элементарные поведенческие описания компилируются в синтезируемые конструкции. И из этого описания компилятор синтезирует опять только связи внутри устройства. А алгоритм работы на все устройство в целом, и в этом случае, будет зависеть от связей и от входных воздействий на данное устройство.

Чем еще отличаются эти языки?

Текст программы на языке программирования пишется для того, чтобы он исполнялся на процессоре. Конечно, программа может быть написана на языке высокого уровня, который сам по себе не зависит от аппаратной платформы. Но компилятор с языка высокого уровня переводит весь текст программы в исполняемый код для определенного целевого процессора, на котором он и выполняется. И в текст программы могут быть добавлены фрагменты, которые позволяют на этапе отладки программы делать какие либо дополнительные действия. Так же можно применить и условную компиляцию и вставить дополнительно фрагменты программы, позволяющие гибко перестраивать текст под нужды программиста. Текст описания аппаратуры на HDL-языке пишется и для того, чтобы провести и симуляцию работы устройства и для того, чтобы получить «прошивку», т.е. загружаемый в FPGA файл. Поэтому в этих языках есть часть языковых конструкций, которые применяются только для симуляции и не могут быть синтезированы в аппаратные узлы. Разработчик обязан следить за тем, что бы при описании того узла, который должен быть загружен в FPGA применялись

только синтезируемые конструкции языка или библиотечные компоненты, поставляемые производителем FPGA.

1.9. Что же общего у этих языков?

Общее - это то, что проекты и в том и в другом языке строятся иерархически. Часть кода выделяется в классы и экземпляры классов объявляются в проекте. Используются библиотеки.

Далее необходимо отметить тенденцию по объединению этих языков. Дело в том, что сейчас появились версии языков программирования SystemC, System Verilog, позволяющие выполнять проекты на языке Си, с последующей автоматической компиляцией этих проектов на языки HDL.

1.10. VHDL и Verilog

«Что же нам надо изучать **VHDL** или **Verilog**?», а ответ такой: «А вы готовы пренебречь половиной рабочих мест только потому, что вы не захотели выучить оба эти языка?»

Нет нужды объяснять, что, как минимум, надо понимать оба этих языка. А на каком языке специализироваться? Есть такое негласное правило: фирма должна делать разработки на том языке программирования, на котором пишет ее лучший программист. Но, поскольку в нашей стране большинство книг написано университетскими преподавателями, а не разработчиками аппаратуры, то имеется явный перекос в сторону VHDL. В то время, как довольно большая часть инженеров с успехом пользуется языком Verilog.

Переход от «картинок» к HDL

Схемное описание проекта, или «картинки» - это наиболее простое и доступное решение для небольших проектов. Оно, несомненно очень наглядно и просто. И на этом все преимущества схемного описания проекта заканчиваются. Можно лишь добавить, что при использовании библиотек компонентов, поставляемых фирмой-изготовителем, удастся применять параметризованные компоненты. Недостатки так же очевидны. При увеличении сложности и объема проекта объем файлов становится очень большим и отладка такого проекта становится очень сложной.

Резюме

HDL-языки по сравнению со схемами:

1. Позволяют делать параметризованные проекты. Не блоки, а именно проекты. Т.е. если Debug=Y, то одни блоки, шины, лог анализаторы и т.д., а если Debug=N, то часть начинки, которая используется только для отладки - удаляется.
2. Из HDL легко сконvertировать блок-схему алгоритма в код и наоборот. Например, используя HDL-дизайнер от фирмы Ментор График. Сразу можно посмотреть и оценить именно то, что сделано.
3. Легче писать комментарии.
4. Легче копировать куски. Меньше трудозатрат с графикой.
5. Правда, надо отметить, что кому-то легче воспринимать картинку, чем текст.

Теперь VHDL и Verilog:

1. Для описания на языке VHDL - приходится набивать больше текста
2. VHDL - более строгий для компиляторов и симуляторов. С Verilog'ом неоднократно

было так, что компиляторы все пропускали, а потом только при синтезе находились ошибки.

Про AHDL:

От него все равно придется уходить на больших проектах, поскольку:

1. Все IP-ядра написаны на HDL и разбираться с ними все равно придется
2. При моделировании нельзя пользоваться файлами для данных, нет возможности пользоваться задачами (Task).

2 Иерархическое описание устройства

Очевидно, что бывают такие схемы, которые не помещаются на одном листе. Или, например, какой то фрагмент схемы повторяется многократно. Что делается в этом случае? Фрагмент схемы выделяется в отдельный чертеж. При этом весь проект становится иерархическим. То-есть, на самом верхнем уровне, чертеж состоит из прямоугольников, которые представляют собой части схемы, выделенные в отдельные чертежи. Те, кто программировал на языке высокого уровня тут же вспомнят вызов функций. Действительно, принцип абсолютно тот же самый. На более высоком уровне описания объекта присутствуют только «главные» характеристики объекта, а все то, что не является соответствующим этому уровню, находится в более низших уровнях описания. То-есть в описании функций или в тех самых отдельных чертежах, о которых шла речь выше. При создании иерархического проекта нас будет интересовать только то, как функционируют «квадратики» - т.е то, что выделено в отдельные блоки и как они подключаются ко всем остальным цепям верхнего проекта.

Теперь надо рассмотреть еще один момент. Если мы выделяем из схемы только один фрагмент и его представляем в виде «квадратика» на верхнем уровне, то тут все понятно. А что происходит, когда надо компактно изобразить на чертеже верхнего уровня, несколько одинаковых фрагментов? Решение здесь точно такое же. Выделяется унифицированный фрагмент схемы и из него делается чертеж нижнего уровня. А на верхнем уровне помещается столько «квадратиков», сколько нам требуется этих унифицированных узлов. Но как же мы узнаем, где и какой фрагмент находится и куда он подключается? А узнаем мы по именам, которые мы даем при установке компонента в схему. Вот теперь мы подошли к объяснению термина «instance» и «instantiate». Фрагмент схемы, выделенный в описание нижнего уровня будем называть «instance» - экземпляр компонента. Размещение на схеме верхнего уровня такого фрагмента в виде блока, будем называть установкой компонента - «instantiate».

Большинство современных языков проектирования аппаратуры поддерживают возможность описания проекта в виде совокупности заранее описанных компонентов и их связей, и язык Verilog не является исключением. Всякая иерархия представлена главным проектным модулем, который называют вершиной проекта, и совокупности подчиненных проектных модулей. Вершина проекта содержит операторы вхождения компонентов. Подчиненный проектный модуль может быть, в свою очередь, вершиной следующей иерархии. В общем случае, проектные модули независимы в том смысле, что каждый может использоваться самостоятельно в различных конструкциях и быть вершиной проекта. Проектные модули, относящиеся к одному проекту, могут находиться в одном файле или представляться несколькими файлами. В последнем случае проектные файлы должны компилироваться в библиотеку проекта в порядке их вхождения в иерархию снизу вверх.

3 Модули (Modules)

Как определяются понятия блоков и субблоков при иерархическом моделировании на Verilog? Язык Verilog имеет термин **модуль**. Модуль - это основной стандартный блок при описании на языке Verilog. Модуль может быть как элементом библиотеки самого нижнего уровня, так и блоком, содержащим несколько субблоков низшего уровня. Как правило, субблоки группируются в модули для того, чтобы обеспечить общие функциональные возможности, которые многократно используются во многих местах проекта. Если проводить аналогию с Си++, то понятие **модуль** в Verilog можно соотнести с понятием **класс** в Си++.

Если рассматривать с точки зрения проекта верхнего уровня, то модуль обеспечивает требуемые функциональные возможности через интерфейс **портов** (входы и выходы), но, при этом, вся «начинка» модуля и ее функционирование будет скрыто для проекта верхнего уровня. Это позволяет проектировщику гибко менять внутреннюю организацию модуля, не затрагивая остальную часть проекта.

В примере ниже показано описание модуля. В языке Verilog, модуль объявляется при помощи ключевого слова **module**. Соответствующее ключевое слово **endmodule** должно быть написано в конце определения модуля. Каждый модуль должен иметь имя - **module_name**, которое является идентификатором для модуля, и список входов и выходов модуля - **module_terminal_list**.

```
module <module_name>
(<module_terminal_list>);
...
<module internals>
...
...
endmodule
```

Пример. Шаблон описания модуля.

Например, триггер типа Т (T-flipflop) может быть определен как модуль, следующим образом:

```
module T_FF (q, clock,
reset);
..
<functionality of T-
flipflop>
..
endmodule
```

Пример . Коды, требуемые для определения триггера типа Т (T-flipflop) как модуля.

Рассмотрим более подробно модули и порты с точки зрения языка Verilog.

Итак, как было сказано выше, модуль - это основной стандартный блок в при моделировании иерархических проектов.

До сих пор не рассматривалась внутренняя организация модулей и описывалось только то, как модули определены и инстанцированы. В этом разделе, мы анализируем

внутреннюю организацию модуля более детально.

Модуль в Verilog состоит из нескольких частей так, как показано на рис. 2.4.

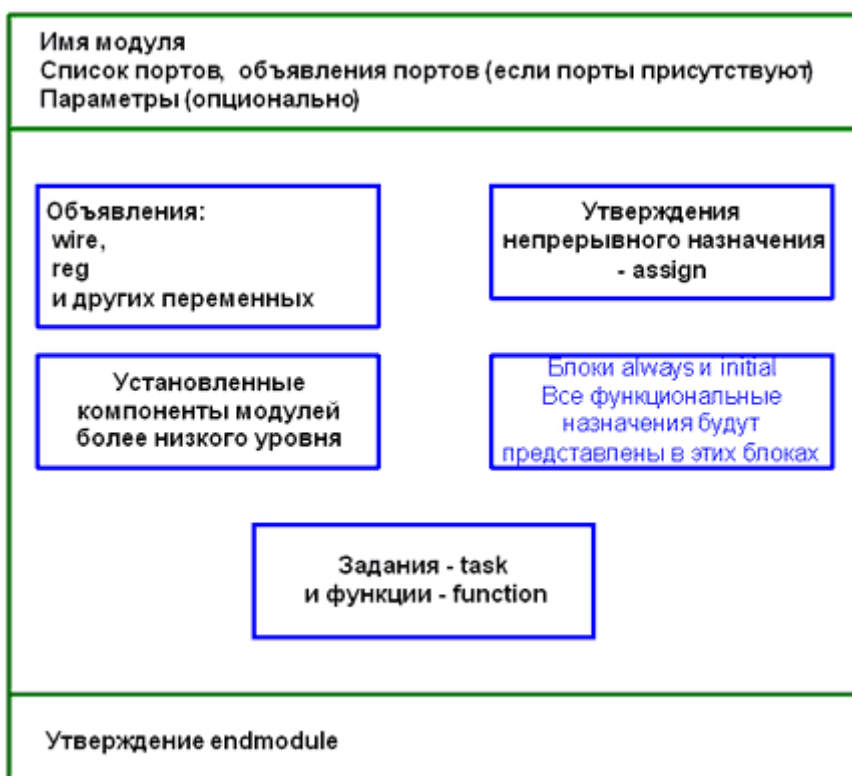


Рисунок . Компоненты модуля в языке Verilog

Определение модуля всегда начинается с ключевого слова - **module**. Название модуля, список портов модуля, объявления портов, и дополнительные параметры должны быть выполнены в начале определения модуля. Список портов и объявления портов имеют место, только в том случае, если модуль имеет какие либо порты. Порты предназначены для того, чтобы взаимодействовать внешними сигналами или другими модулями.

У модуля есть пять компонентов:

- объявления переменных (variable declarations),
- операторы прохождения данных (dataflow statements),
- установленные компоненты модулей (инстансы) более низкого уровня (instantiation of lower modules),
- блоки с поведенческим описанием (behavioral blocks),
- и задачи или функций (tasks or functions).

Эти компоненты могут быть расположены в любом порядке и в любом месте в описании модуля. Слово **endmodule** должно всегда заканчивать определение модуля. Все компоненты (включая объявления переменных) кроме ключевого слова - **module**, названия модуля, и **endmodule** являются дополнительными и могут быть расположены в любом месте в описании модуля в соответствии с тем, как это нужно разработчику при описании модуля.

Verilog позволяет выполнить несколько описаний модулей в одном файле. В таком файле модули могут быть расположены в любом порядке, но производители ПЛИС в описаниях к своим САПР рекомендуют все же выделять каждый модуль в отдельный файл.

Модуль можно назвать шаблоном, из которого Вы можете создать фактические экземпляры модулей (компоненты), устанавливаемые в проект. Когда создается экземпляр модуля, компилятор производит вызов файла модуля и создается новый объект - уникальный экземпляр модуля из шаблона модуля. Каждый объект имеет свое собственное уникальное имя, свои переменные, параметры, и интерфейс ввода - вывода. Процесс создания объектов от шаблона модуля называют установкой (instantiation) компонента, и сами объекты называют компонентами (instance). В примере ниже, в блоке верхнего уровня создаются четыре компонента из одного шаблона T-flipflop (T_FF). Каждый компонент T_FF содержит триггер D_FF и инвертирующий вентиль. Каждому из этих четырех компонентов дано уникальное имя.

```
// Define the top-level module called ripple carry
// counter. It instantiates 4 T-flipflops. Interconnections are
// shown in Section 2.2, 4-bit Ripple Carry Counter.
module ripple_carry_counter(q, clk, reset);
output [3:0] q; //I/O signals and vector declarations
//will be explained later.
input clk, reset; //I/O signals will be explained later.
//Four instances of the module T_FF are created. Each has a unique
//name.Each instance is passed a set of signals. Notice, that
//each instance is a copy of the module T_FF.
T_FF tff0(q[0],clk, reset);
T_FF tff1(q[1],q[0], reset);
T_FF tff2(q[2],q[1], reset);
T_FF tff3(q[3],q[2], reset);
endmodule
// Define the module T_FF. It instantiates a D-flipflop. We assumed
// that module D-flipflop is defined elsewhere in the design. Refer
// to Figure 2-4 for interconnections.
module T_FF(q, clk, reset);
//Declarations to be explained later
output q;
input clk, reset;
wire d;
D_FF dff0(q, d, clk, reset); // Instantiate D_FF. Call it dff0.
not n1(d, q); // not gate is a Verilog primitive. Explained later.
endmodule
```

Пример . Объявление компонентов в проекте.

В языке Verilog, нет возможности делать вложенные описания модулей. Одно определение модуля не может содержать другое определение модуля в пределах определений **module** и **endmodule**. Вместо этого описание одного модуля может включать установленные компоненты от других модулей. Важно не путать описание модуля и установленные компоненты – копии этого модуля. Определение модуля просто описывает то, как модуль будет работать, его внутреннюю организацию и его интерфейс.

Для того, чтобы использовать модуль в проекте необходимо выполнить установку компонента-копии этого модуля и в самом начале файла необходимо вставить директиву препроцессора ``include` с именем того файла, в котором находится описание устанавливаемого модуля.

Синтаксис: ``include имя_файла`. Например, ``include ripple carry.v`.

Отметим, что многие САПР ПЛИС не требуют явного присутствия директивы ``include`, а производят поиск нужного модуля во всех файлах, включенных в проект. Однако если разработчик хочет быть уверенным в переносимости кода, то лучше использовать эту директиву.

2.12.2. Макромодули

Применение макромодуля делает моделирование более эффективным, выполняя определение макромодуля в соответствии с определением вызывающего (родительского) модуля. Однако, компилятор HDL обрабатывает конструкцию макромодуля как же как и конструкцию модуля. Используете ли Вы модуль или макромодуль, в процессе синтеза, будет создана та же самая иерархическая конструкция. На примере показано то, как использовать конструкцию макромодуля.

```
macromodule adder (in1,in2,out1);  
  
input [3:0] in1,in2;  
  
output [4:0] out1;  
  
assign out1 = in1 + in2;  
  
endmodule
```

Пример Использование конструкции макромодуля.

3.1 Порты

Порты обеспечивают интерфейс, которым модуль может общаться с внешней средой. Например, входы ввода – вывода сигналов из микросхемы представляют собой ее порты. Среда может взаимодействовать с модулем только через порты модуля. Внутренняя организация модуля не доступна извне. Это обеспечивает проектировщику очень большие возможности, потому что внутренняя организация модуля может быть изменена, но это не затрагивает внешнюю, по отношению к модулю, среду до тех пор, пока интерфейс модуля не будет изменен.

Порты модуля определяются следующим образом: после имени модуля в круглых скобках приводится список портов модуля так, как показано здесь:

```
module name ( port_list );
```

Определение модуля содержит список портов. Если модуль не обменивается сигналами со средой, то у него нет списка портов.

Порт в списке портов может быть выражен следующим образом:

- Идентификатор
- Единственный бит, выбранный из вектора, объявленного в пределах модуля
- Группа битов, выбранных из вектора, объявленного в пределах модуля
- Конкатенация любого из вышеупомянутых

(Конкатенация - процесс объединения нескольких одно-битовых или много-битовых операндов в один большой битовый вектор. Для получения дополнительной информации, см. раздел “Оператор конкатенации”).

Каждый порт в списке портов должен быть объявлен как ввод, выход, или двунаправленный порт при помощи ключевых слов **input**, **output**, или **inout**.

Рассмотрим 4-битовый полный сумматор, который установлен в проекте верхнего уровня. Диаграмма для портов ввода - вывода показана на рис. 2.5.

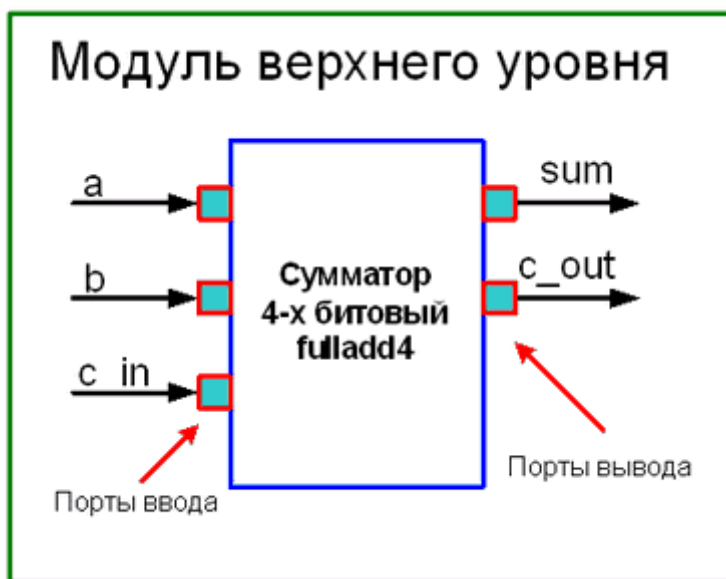


Рисунок Порты ввода - вывода для полного сумматора, установленного в проекте верхнего уровня. У модуля верхнего уровня нет собственных портов.

Обратите внимание на то, что проект **Top** – это проект верхнего уровня. На модуль fulladd4 входные сигналы поступают на порты **a**, **b**, и **c_in**, а выходные сигналы, формируемые модулем, выдаются из него через порты **sum** и **c_out**. Модуль **Top** - модуль верхнего уровня в моделировании и к нему нет необходимости передать или получать сигналы извне. Таким образом, этот модуль не имеет списка портов. Названия модуля и списки портов для обоих объявлений модулей в Verilog приведены в примере.

```
module fulladd4(sum, c_out, a, b, c_in); //Module with a list of ports
module Top; // No list of ports, top-level module in simulation
```

Пример Список портов для модулей Top и fulladd4.

3.2 Параметры (Parameters)

В языке Verilog параметры не принадлежат к какому либо регистру или группе цепей, а, чаще всего, являются атрибутами модуля. Параметры не переменные, они - константы. Параметры могут быть как местные, локальные, так и глобальные. Синтаксис для

объявлений параметров показан в примере

```
<parameter_declaration>
 ::= parameter
<list_of_assignments> ;
```

Пимечание: В некоторых случаях при использовании параметров, можно определять диапазон при объявлении параметра.

<list_of_assignments> - список назначений, разделенный запятыми, где правая сторона назначения должна быть постоянным выражением, то есть, выражение, содержащее только постоянные числа и предварительно определенные параметры. В следующем примере показано объявление параметров.

```
parameter msb = 7; // defines msb as a constant value 7
parameter e = 25, f = 9; // defines two constant numbers
parameter r = 5.7; //declares r as a 'real' parameter
parameter byte_size = 8, byte_mask = byte_size - 1;
parameter average_delay = (r + f) / 2;
```

Хотя параметры в Verilog представляют собой константы, они могут быть изменены во время компиляции для того, чтобы принять новые значения, которые отличаются от определенных в назначении объявления. Это позволяет пользователю настраивать экземпляры модулей при их установке в проекте. Вы можете изменить параметр утверждением defparam, или Вы можете изменить параметр при объявлении экземпляра модуля. Обычно параметры используются для задания значения времени задержки или разрядности (ширины) переменных. Ниже показан пример изменения параметра при объявлении экземпляра модуля.

```
module adder(in1, in2, out)
parameter w = 8;
...
endmodule
```

```
adder ad1 (a, b, summ);
defparam ad1.w = 4;
```

4 Основные конструкции языка

4.1 Соглашения о именовании, комментарии

Для Verilog приняты следующие соглашения об именах, применяемых пользователями в своих проектах:

- Verilog отличает строчные и прописные буквы (case sensitive)
- В именах могут использоваться цифры, буквы, символ подчеркивания - «_» и символ «\$»
- Имена могут начинаться с цифры, буквы или символа подчеркивания - «_»

- В именах нельзя использовать символ пробела

Комментарии в Verilog выглядят так же как и в C: двойная косая черта — строчный комментарий, косая черта со звездочкой — комментарий на несколько строк.

```
-- Эта строка – комментарий
/* Все
   эти
строки
закомментированы
*/
```

Пример Комментарий

Особенностью HDL является то, что в комментарии иногда вставляют директивы, управляющие процессом синтеза, например:

```
// synopsis -translate off
```

4.2 Числа

Постоянные числа могут быть определены в десятичном (**d** or **D**), шестнадцатеричном (**h** or **H**), восьмеричном (**o** or **O**), или в двоичном (**b** or **B**) формате.

Они могут произвольно начинаться со знака + или -, и могут быть написаны в одном из следующих форматов.

1. `<size>'<base><number>` : самая полная форма для записи числа
(`<разрядность>'<основание><значение>`)
2. `<base><number>` : эта форма записи использует разрядность по умолчанию и является машинно-зависимой, но по крайней мере равняется 32 битам
`<number>` : эта форма записи использует разрядность по умолчанию для десятичного
3. формата

Разрядность числа **всегда** указывается в количестве **двоичных** разрядов.

Наиболее простой формат №3 - беззнаковое десятичное число, содержащее, в любом порядке, последовательность цифр от 0 до 9. Хотя пользователь, возможно, не определил размерность написанного числа, Verilog сам вычисляет разрядность числа, примененного в выражении. Обычно в выражениях, разрядность примененных в операторах чисел одинакова. И для использования выбирается требуемое число битов, начиная с наименьшего значащего бита.

Формат №1 определяет постоянный размер для разрядности и записывается следующим образом:

```
aa...a'sf nn...n
```

При этом, неизвестные и высоко-импедансные значения можно использовать во всех числах, кроме десятичных. В каждом случае, применение символов **x** или **z** представляет данное число битов, имеющих состояние **x** — **неизвестное значение** или **z** — **высокий импеданс**. При записи чисел, кроме **z**, может быть использован символ **?**, что применяется в операторе **case**, для того, чтобы улучшить читаемость текста.

Если мы производим операцию над двумя числами, и одно из них имеет меньшую разрядность, чем другое, то в таком случае, для меньшего числа производится заполнение старших разрядов нулями так, чтобы оба числа имели одинаковую разрядность. Если же в числе с большей разрядностью старшие разряды заполнены значениями **x** или **z**, то и число с меньшей разрядностью тоже будет заполнено значениями **x** или **z**.

Символ подчеркивания может вставляться в числа с любым основанием для того, чтобы улучшить читаемость кода. Но, конечно, символ подчеркивания не может быть первым символом в записи числа.

Вот примеры двоичных чисел:

12'b 0x0x_1101_0zx1 и 12'b 0x0x11010zx1

Как мы видим, первое число читается значительно легче.

А это пример неправильной записи:

8'b_0001_1010 - это пример неправильной записи.

Примеры записи чисел, в которых не определена разрядность:

792 - десятичное число

'h 7d9 - шестнадцатеричное число

'o 7746 - восьмеричное число

В примере показаны формы записи чисел, в которых определена разрядность.

```
12'h x // 12-ти битовое число с неопределенным значением
8'h fz // 8-им разрядное шестнадцатеричное число, эквивалентное 8-им
разрядному двоичному числу 8 'b 1111_zzzz
10'd 17 // 10-ти разрядное число, равное 17
```

Пример Примеры записи чисел, в которых определена разрядность

В примере показаны формы записи чисел со знаком и отрицательных чисел.

```
-6 'd 5 // 6-ти разрядное число, равное -5 (т.е., 111011)

6 'd -5 // такая запись - неправильна!

3 'sd 7 // знаковое число, имеющее разрядность в 3 бита, равное -1. т.е.,
децимальное число 7 представленное тремя битами, как 111. Буква s указывает на
то, что число представлено как знаковое (signed), соответственно, значение
такого числа равно минус 1.

4 'sh F // знаковое число, имеющее разрядность в 4 бита, равное -1 т.е.,
шестнадцатеричное число F представленное четырьмя битами, как 1111. Буква s
указывает на то, что число представлено как знаковое (signed), соответственно,
значение такого числа равно минус 1.
```

Пример Примеры записи чисел со знаком и отрицательных чисел.

В этом примере показаны формы записи чисел записи чисел со знаком ?.

```
12'd? // 12-ти битовое десятичное число с записью высокого импеданса как 'don't-care'
```

Пример Примеры записи чисел со знаком ?.

Вещественные числа могут быть записаны в десятичном или научном формате. Если число записано в десятичном формате, оно должно иметь по крайней мере одну цифру после десятичной точки, см. пример.

```
1.8
3_2387.3398_3047
3.8e10 // e or E для основания экспоненты
2.1e-9
3. // неправильная запись
```

Пример Примеры записи вещественных чисел которые могут быть записаны в десятичном или научном формате.

Преобразование вещественного числа в целое число

Язык Verilog преобразовывает вещественные числа в целые числа, округляя вещественное число к самому близкому целому числу, а не отбрасывая десятичную часть. Например, вещественные числа 35.7 и 35.5 и округляются до значения 36, а 35.2 становятся 35. Неявное преобразование имеет место, когда Вы назначаете вещественное число как целое число (*real to an integer*). Вещественные числа не входят в синтезируемое подмножество языка.

4.3 Строки

Строка - это последовательность символов, ограниченная слева и справа кавычками. Они должны быть расположены на одной строке. В строке могут быть использованы специальные ESC-символы, для чего необходимо применять следующим образом символ “\”:

- \n печатать текст с новой строки. Аналогично применению **клавиши RETURN**.
- символ табуляции \t. Эквивалентно тому, что происходит при нажатии на клавишу табуляции.
- Применение двух символов \\ является эквивалентом применения символа \.
- \” является символом “
- \ddd - символ ASCII, определяющий цифру. (в этой записи использовано от одной до трех восьмеричных цифр)
- %% это печать знака %

В примере 2.14 показаны правильная и неправильная формы записи строки.

```

"hello world"; // правильно написанная строка
"good
b
y
e
wo
rld"; // неправильно написанная строка

```

Пример 2.14. Примеры записи строки. Правильная и неправильная формы записи.

4.4 Типы данных

Основная форма представления информации с которой оперирует Verilog — это сигналы, отображающие состояние компонентов и цепей в описываемом устройстве. Кроме того, в проекте могут присутствовать служебные данные, служащие для задания конфигурации, управления моделированием и синтезом, а также отображения результатов моделирования. К служебным типам данных относятся время, действительные и целые числа, строки. Правда числовые значения в некоторых случаях могут сопоставляться сигналам. Язык Verilog не предоставляет возможностей объявления новых типов данных.

Цепи и регистры

Сигналы в Verilog HDL представляются в четырехзначном алфавите {0, 1, X, Z}. Однако для декларации линий связи, к которым подключаются источники, характеризующиеся специфическими электрическими параметрами, например проходные ключи, схемы с открытым коллектором и т.п., драйверам сигнала могут быть приписаны дополнительные атрибуты - уровни силы (Strength Level) - которые доопределяют способ вычисления истинного значения сигнала на линии.

Введено две группы типов данных для представления сигналов: цепи (nets) И регистры (registers). Каждой группе соответствуют специфические способы сохранения присвоенных значений и, в конечном счете, различные схемные решения для их реализации в аппаратуре.

Цепи представляют физические связи между структурными компонентами устройства. Сама по себе цепь не сохраняет состояние - она должна управляться драйвером, который соответствует оператору параллельного присваивания, причем имя драйвера совпадает с именем цепи. Это, собственно, и отражается в принятом в языке Verilog названии операторов этого типа - "continuous", т.е. непрерывные, постоянно воздействующие на цепь. Ключевое слово для объявления цепей — wire.

Регистр - это обобщенное понятие, отражающее элементы, способные сохранять состояние. Программная модель предусматривает сохранение состояния от каждого присвоения до следующего. Иными словами, оператор присваивания значения регистру действует как сигнал установки нового значения. В Verilog определен широкий набор конструкций для задания условий изменения состояний регистров и представления различных способов управления триггерными устройствами.

Различают несколько типов цепей: wire, wand, wor, supply0, supply1, tri, tri0, tri1, triand, trior, trireg. Типы цепей wire и tri с точки зрения программной интерпретации эквивалентны. Различные обозначения служат лишь для лучшей читаемости программы. Обычно тип wire используется для цепей, подключенных к единственному источнику, а тип tri - для представления связей, управляемых несколькими источниками (драйверами).

Прочие типы цепей служат для представления различных вариантов монтажной и коммутационной логики, которые не характерны для ПЛИС и далее рассматриваться не будут.

Если в декларации отсутствует объявление диапазона, то соответствующее имя присваивается скалярной переменной, представляющей состояние одновыходного логического элемента или одной линии соединений. В противном случае предполагается объявление битового вектора, т. е. кода, число разрядов которого и порядок их нумерации задается конструкцией [<индекс старшего разряда>:<индекс младшего разряда>]. Право доступа определяет, возможен ли доступ к отдельным компонентам вектора, т. е. можно ли присваивать значения отдельным разрядам кода и использовать эти значения в других выражениях. Если перед указанием диапазона записано ключевое слово *vectored*, то доступ к отдельным компонентам вектора не допускается. Если присутствует ключевое слово *scalared* (а также по умолчанию), каждый компонент кода может устанавливаться и анализироваться индивидуально.

В выражениях языка можно обращаться и использовать как один бит, так и несколько соседних битов векторного регистра (или шины). Выбор одного бита из регистра или шины называют **выбором бита** (*bit-select*), и выбор нескольких соседних битов называется **выбором группы** (*part-select*).

Следующий пример показывает выбор бита из **acc**, этот бит адресуется операндом **index**:

```
acc[index]
```

Выбор части битов из регистра или цепи показан на следующем примере:

```
vect[ms_expr:ls_expr]
```

Форма записи регистров показана в примере .

```
reg [10:0] counter; reg a; reg [2:0] b;
reg [-5:7] c
a = counter [7]; // бит номер 7 из counter загружен в a
b = counter [4:2]; // биты номер 4, 3, 2 из counter загружены в b
```

Пример Примеры записи регистров.

Для выбора бита, можно пользоваться выражениями или литералами. Группы битов, выбранные как часть регистра или шины должны быть определены выражениями или литералами, имеющими определенную разрядность и эти значения могут быть положительными или отрицательными.

Как регистр, так и цепь могут быть объявлены как знаковые, например:

reg signed [15:0] My16BitReg

Такая форма записи указывает на то, что при работе с этим регистром данные должны быть обработаны как число со знаком (двоичное дополнение). Ответственность за корректность обработки таких чисел лежит полностью на системе синтеза(моделирования). Отметим, что не так давно системы синтеза не поддерживали знаковые числа, т.е. просто игнорировали определение signed.

Памяти

Память описывается как массив регистров. Вот синтаксис для описания такого массива:

`reg [<memory_width>] <reg_name> [<memory_depth>];`

Блоки памяти определяются, используя объявление регистра, см. пример

```
reg [10:0] lookUpTable [0:31];
```

Пример Пример объявления блока памяти.

Так определяется массив из 32-х слов, названный **lookUpTable**, где каждое слово имеет разрядность в 11 битов. Память используется в выражениях следующим образом, например,

`lookUpTable [5] = 75;`

Эта запись показывает, что пятое слово в памяти **lookUpTable** загружено значением 75. Блоки памяти могут быть многомерными. Вот так объявляются двух- и трехмерные массивы, см. пример

```
reg [7:0] twoDarray [15:0] [63:0];  
reg [9:0] threeDarray [31:0] [31:0] [43:0];
```

Пример Примеры объявлений двух- и трехмерных массивов.

Если 8-битовый регистр **a_reg** уже объявлен, то запись данных в этот регистр выглядит следующим образом:

`a_reg = twoDarray[3][2];`

Группы битов, выбранные как часть регистра или шины и выбор бита могут быть применены и к блокам памяти. Чтобы выбрать часть битов памяти, к ней, в конце записи, добавляется еще один индекс. Таким образом, выбор битов от бита 7 и до бита 0 из памяти **threeDarray** может быть записан так, см. пример

```
a = threeDarray [21] [4] [40] [7:0];
```

Пример Выбор битов от бита 7 и до бита 0 из памяти **threeDarray**.

Чтение элементов массива памяти вне диапазона, определенного для чтения, возвращает неизвестное значение - **x**. Запись в элементы массива памяти вне диапазона, определенного для записи, не имеет никакого эффекта. Выбрать одновременно больше чем одно слово из памяти невозможно.

Целые числа и время

Регистры используются для того, чтобы моделировать аппаратные средства. Иногда бывает полезно выполнить дополнительные вычисления в целях моделирования. Например, мы можем захотеть выключить какое либо управление в проверяемом узле после того, как прошло определенное время при симуляции. Если мы используем регистры с этой целью то, операции на них могут быть перепутаны с действиями фактических аппаратных средств.

Целые числа-Integer и переменные **времени - time** обеспечивают возможность для описания вычислений, требующихся для моделирования. Они делают описание более наглядным и самодокументируемым и облегчают сопровождение проектов. Для объявления целых чисел используется ключевое слово **integer** и далее идет список переменных. Объявление времени -

то же самое за исключением, того, что используется ключевое слово *time*, см. пример

```
integer a, b;      // два целых числа
integer c [1:100]; // массив целых чисел
time q, r;         // две переменные для времени
time s [31:0];     // массив времен
```

Пример Объявления целых чисел и времени.

Целое число – обычно имеет разрядность в 32 бита. Операции с целым числом обычно выполняются в двоичном коде с дополнением, поэтому старший бит указывает признак знака. Переменная времени – это обычно 64-х битовая переменная, используемая с системной функцией \$time о которой будет сказано ниже.

4.5 Операторы

Операторы могут быть трех типов: унитарные, бинарные, и тернарные.

Унарные операторы производят действия над единственным операндом.

Бинарные операнды производят действия над двумя операндами.

Тернарные операторы имеют два оператора, которые делают выбор для трех операндов.

Примеры этих операндов:

`a = ~ b;` // ~ унитарный оператор. **b** – это операнд

`a = b && c;` // && бинарный оператор. **b** и **c** – операнды

`a = b ? c : d;` // ?: тернарный оператор. **b**, **c** и **d** – операнды

В языках VHDL и Verilog используются следующие арифметические и битовые операторы:

Арифметические(Arithmetic),

Конкатекации (объединения) (Concentration)

Повторения (Replication),

Условные (Conditional),

Равенства (Equality),

Логические по-битовые операторы (Logical Bit-wise),

Логического сравнения (Logical Comparison),

Свертки (Reduction),

Отношения (Relational),

Сдвига (Shift),

Унарные арифметические операторы (Unary Arithmetic (Sign)).

Операнды (Operands)

Есть несколько типов операндов, которые могут быть определены в выражениях.

Самый простой тип – это цепь или регистр, и, если при этом указывается на полную разрядность цепи или регистра, то можно приводить только название цепи или регистра. В этом случае, все биты, составляющие цепь или выходы регистра используются как операнд.

Как цепи, так и регистры могут иметь несколько разрядов. При этом, если используется только единственный бит из вектора цепи или регистра, то необходимо использовать операнд выбора бита. Если при обращении к группе смежных битов в векторной цепи или в регистре, используется только часть битов, то необходимо применить операнд выбора части битов из вектора.

На элемент памяти можно сослаться как операнд.

Конкатенация других операндов, (включая вложенные конкатенации) может быть определена как операнд.

Запрос функции - это тоже операнд.

Операторы и возможность их применения в VHDL или в Verilog приведены в таблице.

Таблица 2-3. Операторы и возможность их применения в VHDL или в Verilog

Операция	Оператор		Описание
	VHDL	Verilog	
Арифметические операторы (Arithmetic Operators) exponential multiplication division addition subtraction modulus remainder absolute value	** * / + - mod rem abs	 * / + - / %	Возведение в степень Умножение Деление (для целочисленных операндов деление целочисленное) Сложение Вычитание Целочисленное деление Остаток от деления Абсолютная величина
Concentration and Replication Operators concentration replication	 &	 { } { { } }	 объединение (конкатенация) повторение (репликация)
Conditional Operator conditional		?:	Условный оператор
Операторы равенства (Equality Operators) equality inequality	= /=	== !=	Равно Не равно
Битовые логические операторы (Logical Bit-wise Operators) unary negation NOT binary AND binary OR	not and or	~ & 	по-битовая инверсия поразрядное двоичное «И» поразрядное двоичное «ИЛИ»

binary NAND binary NOR binary XOR binary XNOR	nand nor xor xnor	 ^ ^ ~ or ~ ^	поразрядное двоичное «И-НЕ» поразрядное двоичное «ИЛИ-НЕ» поразрядное двоичное «Исключающее ИЛИ» поразрядное двоичное «Исключающее ИЛИ-НЕ»
Операторы логического сравнения (Logial Comparison Operators) NOT AND OR	not and or	! && 	поразрядное логическое «НЕ» поразрядное логическое «И» поразрядное логическое «ИЛИ»
Операторы свертки (Reduction Operators) AND OR NAND NOR XOR XNOR		 & ~& ~ ^ ^ ~ or ~ ^	Побитовая свертка
Операторы отношения (Relational Operators) less than less than or equal to greater than greater than or equal to	< <= > >=	< <= > >=	Меньше Меньше или равно Больше Больше или равно
Операторы сдвига (Shift Operators) logical shift left logical shift right arithmetic shift left arithmetic shift right logical rotate left logical rotate right	sll srl sla sra rol ror	<< >> 	Логический сдвиг влево Логический сдвиг вправо Арифметический сдвиг влево Арифметический сдвиг вправо Циклический сдвиг влево Циклический сдвиг вправо
Идентичность		=== !==	Идентично Не идентично

Таблица 2-4. Приоритеты выполнения операторов.

Приоритет выполнения операторов	
! ~ * / % + - << >> < <= > >= == != === !== & ^ ^~ 	высший приоритет исполнения

&& ?: (ternary operator)	низший приоритет исполнения
-----------------------------------	------------------------------------

Обратим внимание на операторы равенства и идентичности. Оператор равенства (==) возвращает 1 (истина), если сравниваемые операнды побитно равны. Если в коде любого из операндов хотя бы один разряд имеет значение 'x' или 'z', то результат сравнения — 'x' (не определено состояние). То же справедливо и для оператора неравенства (!=). Иначе обстоит дело с операторами идентичности (===, !==), здесь, если в сравниваемых кодах в одинаковых позициях находятся одинаковые значения, в том числе 'x' и 'z', то коды считаются равными. Результат операций «===» и «!==» может быть только 0, либо 1.

Побитовые операторы (Bit-Wise Operators)

Битовые операторы позволяют производить по-битовые операции над двумя или более операндами. При этом операция применяется отдельно к каждому разряду (или к каждой паре разрядов) операнда(-ов).

Таблица 2-5. Поразрядная инверсия.

~	
0	1
1	0
x	x

Таблица 2-6. Поразрядная операция «И» (AND).

&	0	1	x
0	0	0	0
1	0	1	x
x	0	x	x

Таблица 2-7. Поразрядная операция «ИЛИ» (Or).

	0	1	x
0	0	1	x
1	1	1	1
x	x	1	x

Таблица 2-8. Поразрядная операция «Исключающее ИЛИ» (exclusive Or).

^	0	1	x
0	0	1	x
1	1	0	x
x	x	x	x

Таблица 2-9. Поразрядная операция «Исключающее ИЛИ-НЕ» (exclusive NOR).

$\wedge \sim$	0	1	x
0	1	0	x
1	0	1	x
x	x	x	x

Необходимо обратить внимание на то, чтобы отличить битовые операторы $\&$ и $|$ от логических операторов $\&\&$ и $||$. Например, если $x = 1$, и $y = 2$, то битовая операция $x \& y$ в результате даст 0, в то время как $x \&\& y$ – в результате даст 1. Когда операнды имеют неравное число битов, то операнд, имеющий меньшее число бит будет заполнен нулями в старших разрядах.

Операторы приведения

Операторы приведения выполняют операции по-битно над операндом, чтобы произвести результат, имеющий разрядность - 1 бит. Первый шаг операции выполняется над первым и вторым битами операнда, в соответствии с таблицами 2-10 - 2-12. Вторые и последующие шаги выполняются аналогично с однобитовым результатом предшествующего шага и следующим битами операнда, т.е. операторы приведения позволяют получить свертку двоичного представления числа по заданному оператору. Операторы приведения имеют такие же обозначения, как и соответствующие поразрядные операции, но операции приведения являются унарными, т.е. применяются только к одному операнду:

Приведение переменной **a** по «И»:

$\& a$

Поразрядное «И»:

$a \& b$

Таблица 2-10. Оператор приведения по «И» (AND).

$\&$	0	1	x
0	0	0	0
1	0	1	x
x	0	x	x

Таблица 2-11. Оператор приведения по «ИЛИ» (Or).

$ $	0	1	x
0	0	1	x
1	1	1	1
x	x	1	x

Таблица 2-12. Оператор приведения по «Исключающему ИЛИ» (exclusive Or).

^	0	1	x
0	0	1	x
1	1	0	x
x	x	x	x

Необходимо отметить, что оператор приведения по «И-НЕ» и оператор приведения по «ИЛИ-НЕ» выполняет те же операции, что и операторы приведения по «И» и «ИЛИ», соответственно, но результаты их работы при этом будут инверсными. Результаты работы операторов приведения приведены в таблице 2-13 и таблице 2-14.

Таблица 2-13. Результаты работы оператора приведения для «И», «ИЛИ», «И-НЕ» и «ИЛИ-НЕ».

Результат работы оператора приведения для &, , ~&, и ~		
Операнд	&	~& ~
биты не установлены в	0 0	1 1
все биты установлены в	1 1	0 0
часть битов, но не все, установлены в	0 1	1 0

Таблица 2-14. Результаты работы оператора приведения для «Исключающего ИЛИ» и «Исключающего ИЛИ-НЕ».

Результат работы оператора приведения для ^ и ~^		
Операнд	^	~^
Значения нечетных битов	1	0
Значения четных битов, если они есть	0	1

Ограничения синтаксиса

В языке Verilog есть два ограничения синтаксиса, предназначенные для того, чтобы защитить текст описания от ошибки, которая возникает при неправильной записи выражения. Эти ошибки бывает достаточно трудно найти, потому что ошибки возникают из-за неправильного положения пробела и символа в выражении. Для примера можно рассмотреть два приведенных ниже выражения. Выражения строк 1 и 2 внешне почти похожи, но дают совершенно разный результат, см. пример

- | |
|--------------------------------------|
| 1. a & &b a b
2. a && b a b |
|--------------------------------------|

Пример

Чтобы защитить пользователей от этого типа ошибок, необходимо использовать круглые скобки для того, чтобы отделить приведение по **или** или **и** (**reduction or or and**) от поразрядного **или** или **и** (**bit-wise or or and**) оператора. В следующем примере показан синтаксис, который требует круглых скобок:

Неправильный синтаксис	Правильный синтаксис
a & &b	a & (&b)
a b	a (b)

Пример 2.2. Примеры неправильного и правильного синтаксиса.

Операторы сдвига

Операторы сдвига, << и >>, выполняют сдвиг операнда, находящегося слева от оператора, влево или вправо на то число позиций двоичного разряда, которая указана справа от оператора сдвига. При сдвиге данных оба оператора производят заполнение освободившихся позиций двоичного числа нулями.. Следующий пример иллюстрирует действия, выполняемые этими операциями.

```
module shift;
reg [3:0] start, result;
initial
begin
start = 1; // Start is set to 0001
result = (start << 2); // Result is set to 0100
end
endmodule
```

Пример работы операторов сдвига

В этом примере четырех-битовому регистру **start** присвоено значение **0001**. После сдвига влево на два разряда регистр будет содержать число **0100**.

Условный оператор

Аналогичен условному оператору Си, имеет три операнда, отделенные двумя операторами, записанными в следующем порядке:

```
cond_expr ? true_expr : false_expr
```

Если **cond_expr** оценивается как ложное - **false**, то для результата будет использовано выражение **false_expr**. Если условное выражение истинно - **true**, то для результата будет использовано выражение **true_expr**.

Если **cond_expr** или **true_expr** и **false_expr** содержат символы неоднозначности - X, то для того, чтобы вычислить конечный результат, используйте таблицу 2-15.

Если разрядность операндов различна, то операнд с меньшей разрядностью будет слева дополнен нулями (старшие разряды) так, чтобы соответствовать разрядности операнда с большей разрядностью.

Таблица 2-15. Работа условного оператора.

?:	0	1	x	z
0	0	x	x	x
1	x	1	x	x
x	x	x	x	x
z	x	x	x	x

В примере 2.5 показано то, как можно использовать условный оператор для того, чтобы выполнить шину с третьим состоянием.

```
wire [15:0] busa = drive_busa ? data : 16'bz;
```

Пример 2.5. Применение условного оператора для того, чтобы выполнить шину с третьим состоянием.

Шина **busa** управляется сигналом **drive_busa**. В том случа, когда этот сигнал 1, на шину передаются данные - **data**, в противном случае, шина переключается в третье состояние. Если значение сигнала **drive_busa** неизвестно, то и значение данных на шине **busa** тоже неизвестно.

Конкатенация - Concatenations

Конкатенация - объединение битов, находящихся в двух или больше выражениях. Для записи конкатенации, используются символы фигурных скобок { и }, с запятыми, отделяющими выражения, находящиеся в этих скобках. Следующий пример 2.6 объединяет четыре выражения:

```
{a, b[3:0], w, 3'b101}
```

Пример 2.6. Применение оператора конкатенации для того, чтобы объединить четыре выражения.

Предыдущий пример эквивалентен следующей записи, см. пример 2.7.

```
{a, b[3], b[2], b[1], b[0], w, 1'b1, 1'b0, 1'b1}
```

Пример 2.7. Применение оператора конкатенации, эта запись эквивалентна примеру 2.6.

Постоянные числа, разрядность которых не указана, не должны быть использованы в конкатенациях, потому что разрядность каждого операнда в конкатенации необходима для того, чтобы вычислить полный размер шины для конкатенации. Конкатенации могут быть записаны, используя множитель повторения так, как показано в примере 2.8.

```
{4{w}} // This is equivalent to {w, w, w, w}
```

Пример 2.8. Применение оператора конкатенации, с использованием множителя повторения.

Репликация

Оператор репликации позволяет повторить небольшой кусок кода заданное число раз. Его синтаксис:

```
{<число повторений>{<повторяемое выражение>}}
```

Этот оператор, с одной стороны проще циклов (хотя выполняет почти те же функции) и, с другой стороны позволяет задавать число повторений параметром, что бывает весьма удобно.

```
b, {3{a, b}} // Эта запись эквивалентна:  
// b, a, b, a, b, a, b
```

Пример применения оператора репликации

5 Назначения (Assignments)

5.1 Краткий обзор Назначений (присваиваний)

Назначение - основной механизм для того, чтобы получить значения для цепи и для регистра. Есть две канонических формы назначения:

- непрерывное назначение, которое назначает значения на цепь
- процедурное назначение, которое назначает значения на регистры

Назначение состоит из двух частей, левой стороны и правой стороны, отделенной символом равенства (=). Правая сторона может быть любым выражением, которое вычисляется до некоторого определенного значения. Левая сторона указывает переменную, на которую должно быть сделано назначение, вычисленное в соответствии с тем, что записано на правой стороне. Левая сторона может принять одну из следующих форм в зависимости от того, является ли назначение непрерывным назначением (**continuous assignment**) или процедурным назначением (**procedural assignment**), см. таблицу 2-19.

Таблица 2-19. Разрешенные формы назначений для левой стороны.

Тип назначения	Выражение на левой стороне
непрерывное назначение	• цепь (net)(vector или scalar) • определенный бит, выбранный из векторной цепи (constant bit select of a vector net) • определенная группа битов, выбранных из векторной цепи (constant part select of a vector net) • комбинация любых вариантов из 3-х перечисленных
	• регистр (register) (vector или scalar) • определенный бит, выбранный из регистра (bit select of a vector register) • определенная группа битов, выбранных из регистра (onstant part select of a vector register) • элемент памяти(memory element) • комбинация любых вариантов из 4-х перечисленных

5.2 Непрерывные присваивания

Непрерывные назначения управляют значениями, назначаемыми на цепь, на вектор и на скаляр. Значение слова "непрерывное" состоит в том, что назначение происходит всякий раз, когда моделирование заставляет значение правой стороны изменяться. Непрерывные назначения обеспечивают способ моделировать комбинационную логику, не определяя взаимосвязь вентилей логики. Вместо этого модель определяет логическое выражение, которое управляет цепью. Выражение на правой стороне непрерывного назначения ничем не ограничено. Оно может даже содержать ссылку на функции. Таким образом, результат оператора выбора - **case**, условного оператора - **if**, или другой процедурной конструкции может управлять цепью.

Синтаксис непрерывного назначения:

```
<net_declaration>
||= trireg <charge_strength>? <expandrange>? <delay>?
<list_of_variables> ;
||= <NETTYPE> <drive_strength>? <expandrange>? <delay>?
<list_of_assignments> ;
<continuous_assign>
::= assign <drive_strength>? <delay>? <list_of_assignments> ;
<expandrange>
::= <range>
||= scaled <range>
||= vectored <range>
<range>
::= [ <constant_expression> : <constant_expression> ]
<list_of_assignments>
::= <assignment> <,<assignment>>*
<charge_strength>
::= ( small )
||= ( medium )
||= ( large )
<drive_strength>
::= ( <STRENGTH0> , <STRENGTH1> )
||= ( <STRENGTH1> , <STRENGTH0> )
```

Подробно отдельные части назначения будут рассмотрены ниже.

5.3 Назначение, выполняемое при объявлении цепи

В Verilog возможно выполнение сетевого назначения, при объявлении цепи. Это позволяет поместить непрерывное назначение на цепь в том же самом утверждении, которое объявляет эту цепь. Следующий пример для непрерывного назначения при `<net_declaration>`:

```
wire (strong1, pull0) mynet = enable;
```

Необходимо отметить: поскольку цепь может быть объявлена только однажды, то только одно назначение на эту цепь может быть сделано при объявлении именно этой конкретной цепи. Это отличается от обычного непрерывного назначения, так как обычно одна цепь может получить многократные

непрерывные назначения. Назначения при объявлении поддерживаются не всеми системами синтеза!

5.4 Оператор непрерывного присваивания - *assign*

Утверждение <continuous_assign> помещает непрерывное назначение на цепь, которая была предварительно объявлена, или явно, в соответствии с объявлением цепи, или неявно при использовании ее названия в списке цепей для примитива- вентиля, определяемого пользователем примитива, или устанавливаемого в проект экземпляра модуля. Вот пример непрерывного назначения на цепь, которая была предварительно объявлена:

```
assign (strong1, pull0) mynet = enable ;
```

Назначения на цепях являются непрерывными и автоматическими. Это означает что всякий раз, когда операнд, находящийся с правой стороны выражения меняет свое значение в течение моделирования, то все, что находится с правой стороны выражения пересчитывается и назначается на левую сторону выражения.

Вот пример использования непрерывного назначения для того, чтобы промоделировать сумматор на четыре бита с переносом, см. пример 2.38.. Отметьте, что назначение не могло быть определено непосредственно в объявлении цепей, потому что оно требует конкатенации в левой части выражения.

```
module adder (sum_out, carry_out, carry_in, ina, inb);
output [3:0]sum_out;
input [3:0]ina, inb;
output carry_out;
input carry_in;
wire carry_out, carry_in;
wire[3:0] sum_out, ina, inb;
assign
{carry_out, sum_out} = ina + inb + carry_in;
endmodule
```

Пример 2.38. Использование утверждения для непрерывного назначения.

Следующий пример 2.39 описывает модуль с одной 16-разрядной выходной шиной. В нем производится выбор между одной из четырех входных шин и выбранная шина подключается к выходной шине.

```
module select_bus (busout, bus0, bus1, bus2, bus3, enable, s);
parameter n=16;
parameter Zee=16'bz;
output [1:n ] busout;
input [1:n] bus0, bus1, bus2, bus3;
input enable;
input [1:2] s;
tri [1:n] data; // net declaration.
tri [1:n] busout= enable ? data : Zee; // net declaration with
// continuous assignment.
assign // assignment statement with
```

```

data = (s==0) ? bus0 : Zee, // 4 continuous assignments.
data = (s==1) ? bus1 : Zee,
data = (s==2) ? bus2 : Zee,
data = (s==3) ? bus3 : Zee;
endmodule

```

Пример 2.39. Назначение на цепь при объявлении и оператор непрерывного назначения.

В примере 2.39 приводится следующая последовательность событий при моделировании описания:

1. Имеется входная переменная **s**, значение которой проверяется при назначении цепи. И в соответствии с тем, какое значение принимает **s**, на шину **data** коммутируются данные от одной из четырех входных шин.
2. Для выходной шины сделано непрерывное назначение в объявлении цепи для установки данных так, что если установлен сигнал разрешения - **enable**, то содержание данных назначено на выходную шину, если же сигнал разрешения не принимает ни одного из значений использованных для сравнения (0, 1, 2, 3, 4), то на выходную шину будет назначено значение **Zee**.

Сила сигнала

Язык Verilog позволяет описывать устройства, в которых несколько источников работают на одну линию, при этом каждому источнику сопоставляется собственный оператор непрерывного присваивания переменной, представляющей сигнал на этой линии. Если при этом лишь один из таких источников может быть в активном состоянии, то ситуация описывается относительно просто: если хоть один источник находится в активном состоянии, переданное им значение, как более "сильное", по определению подавляет "слабые" сигналы, представляющие источники, находящиеся в отключенном состоянии (z-состоянии). Однако на практике возникают и иные ситуации. Во-первых, может быть необходимо обнаружение при моделировании сбойных ситуаций, связанных с некорректной активизацией нескольких драйверов. Во-вторых, возможно подключение к шинам схем с открытым коллектором и других компонентов, выходное сопротивление которых, а значит влияние на результат, неодинаково в различных условиях. Конструкция, определяющая степень влияния драйвера на результат, называется "Силой драйвера", и может включаться в оператор параллельного присваивания.

Сила драйвера задается парой значений уровня силы, выбранных обязательно из разных столбцов приведенного списка:

supply1	supply0
strong1	strong0
pull1	pull0
weak1	weak0
highz1	highz0

Значения, заканчивающиеся символом 1, определяют уровень силы драйвера в состоянии логической единицы, а заканчивающиеся символом 0, - в состоянии логического

нуля. Порядок записи безразличен. Если на сигнал действует несколько драйверов, имеющих различный уровень силы, то сигналу присваивается значение того драйвера, который в данный момент представлен наиболее сильным сигналом (значения в приведенном списке упорядочены, причем выше в списке представлены более сильные значения). Если подключены равносильные драйверы, сигналу присваивается неопределенное значение. Пример и временные диаграммы на рисунке иллюстрируют использование признака силы драйвера.

```

wire a;
reg b,c;
assign (strong0,weak1) a=b;
assign (strong1,weak0) a=c;
initial
  begin c=1'bz; b=0;
    #10 c=1;
    #10 b=1;
    #10 c=0;
    #10 c=1;
    #10 b=1'bz;
  end;

```

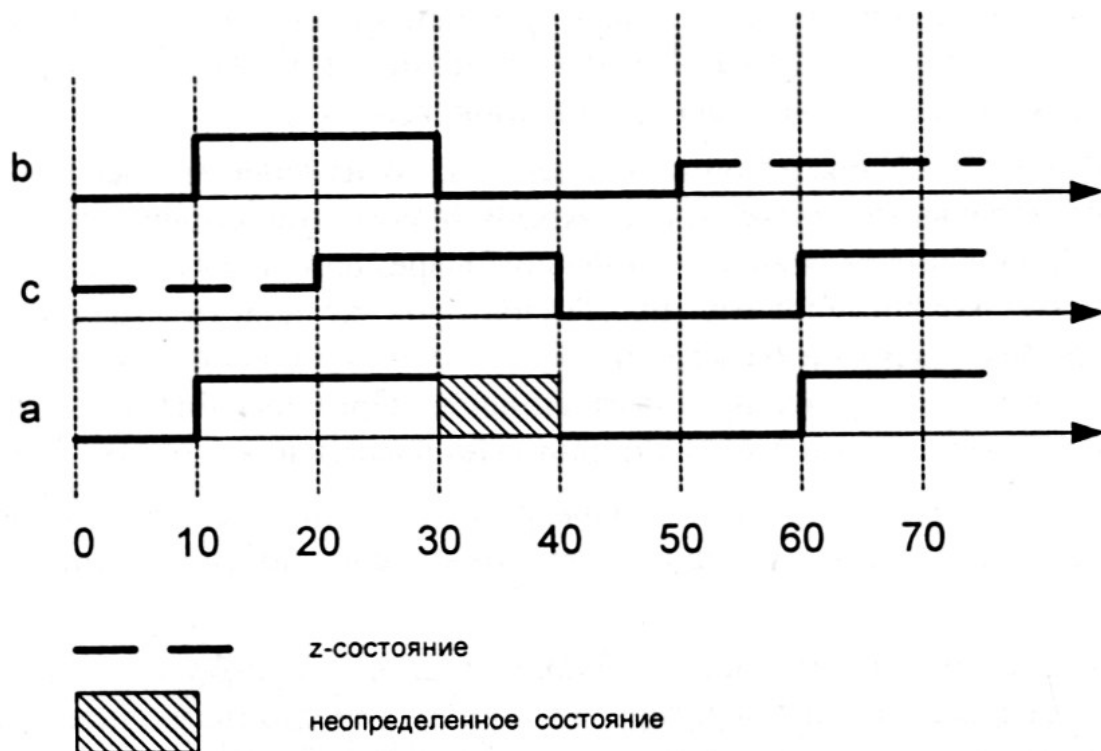
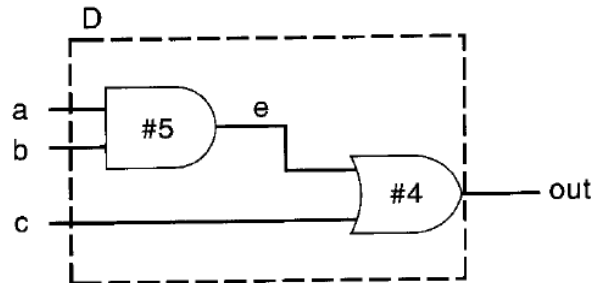


Рис. 3.22. Влияние уровня силы драйвера на значение сигнала

5.5 Задержки Delays

Задержка (delay), заданная при непрерывном назначении, определяет продолжительность времени между изменением значения, находящегося справа от операнда и назначением, сделанным на левую сторону операнда. Например схему представленную на рисунке ниже



можно описать следующим образом:

```
module D (out, a, b, c);  
  
  // Порты  
  output out;  
  input a,b,c;  
  
  // Внутренняя цепь  
  wire e;  
  
  assign #(5) e = a & b; //Delay of 5 on gate a1  
  assign #(4) out = e & c; //Delay of 4 on gate o1  
  
endmodule
```

Если то, что находится слева от операнда, представляет собой скалярную цепь, то задержка будет представлять собой такую же задержку, какая имеет место в примитивах - вентилях, то есть производится задержка того сигнала, который приходит на вентиль, причем можно дать различные задержки для нарастающего фронта сигнала, спадающего фронта сигнала, и перехода сигнала в высокий импеданс.

Если то, что находится слева от операнда присваивания, представляет собой векторную цепь, то могут быть применены до трех различных задержек: задержка переключения сигнала из 0 в 1 (rise), из 1 в 0 (fall) и переключения в z-состояние (turn-off). Синтаксис такого назначения приведен ниже:

```
assign #(<rise>,<fall>,<turn-off>) <имя цепи> = <выражение>
```

Следующие правила определяют, которая из задержек будет выполняться при назначении:

- Если правая сторона была ненулевой и становится нулем, то используется задержка для спадающего фронта сигнала.
- Если правая сторона становится z, то используется задержка перехода сигнала в высокий импеданс.
- Для остальных случаев используется задержка для нарастающего фронта сигнала.

В случае, если указано только одно значение задержки, то оно используется для всех типов переходов. Если указано две задержки, то они используются для переходов 0-1 и 1-0 (соответственно), при этом для переключения в z будет использована наименьшая из них.

Примеры определения задержек:

```
assign #(5) a1 = i1 & i2; //Задержка 5 для всех переходов
assign #(4,6) a2 = i1 | i2; // Rise = 4, Fall = 6
assign #(3,4,5) b0 = in & control; // Rise = 3, Fall = 4, Turn-off = 5
```

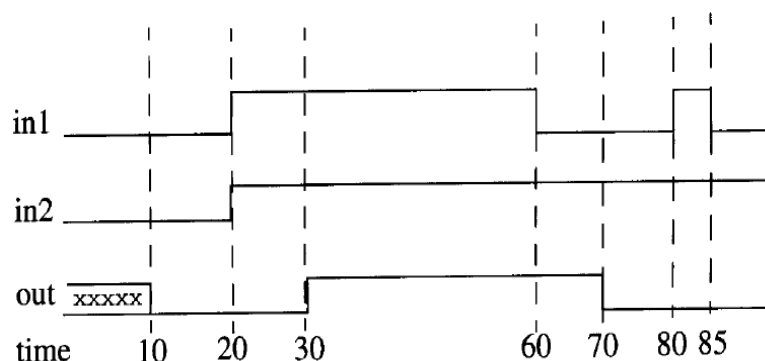
Если производится объявление цепи, то непосредственно в этом же объявлении можно назначить и задержки на эту цепь. Такое назначение задержки отличается от того случая, который имеет место в объявлении задержки при непрерывном назначении на цепь. Значение задержки может быть применено к цепи в объявлении цепи так, как показано в следующем примере:

wire #10 wireA;

Такая форма записи применяется для того, чтобы показать, что в цепи, называемой **wireA** имеется задержка в течение десяти единиц времени, прежде чем сигнал, передаваемый по цепи **wireA**, пройдет через эту цепь и поступит для моделирования во все утверждения, где участвует данный сигнал. В том случае, когда задержка делается при непрерывном назначении в объявлении, эта задержка является только частью непрерывного назначения и не является задержкой всей цепи. И в этом случае, она не будет добавлена к задержке других драйверов на этой цепи. Более того, эта задержка не добавляется в том случае, если производится назначение, приводящее к расширению векторной цепи (т.е. добавляются цепи, которые не входили в первоначальное определение с ключевым словом **vector**), и в этом случае задержки на положительный фронт и задержки на отрицательный фронт не будут применены к индивидуальным битам, если назначение включено в объявление этих цепей.

Вообще говоря, задержки бывают двух типов: инерционная и транспортная. Представим ситуацию, когда цепь имеет задержку. В этом случае, если операнд, находящийся с правой стороны изменяется прежде, чем прошло время, необходимое для прохождения сигнала через эту цепь от предыдущего изменения, то, тогда будет обработано только одно последнее изменение значения. Таким образом, произойдет только одно назначение на цепь. Иначе говоря импульс, длительность которого меньше задержки срабатывания узла не будет обработан. Этот эффект называется инерционная задержка. При использовании задержки транспортного типа все изменения входного сигнала, какими бы кратковременными они ни были отражаются на работе схемы. Язык Verilog (в отличие от VHDL) не имеет встроенных средств для моделирования транспортной задержки. Все задержки носят инерционный характер. Проиллюстрируем это примером:

```
assign #10 out = in1 & in2; // Delay in a continuous assign
```



Как видно из временной диаграммы импульс на 80-ом моменте времени не был воспринят схемой.

В следующем примере 2.40 осуществляется обработку вектора по исключающему ИЛИ. Разрядность шин и задержка в этом примере определены как параметры, они могут быть изменены, когда экземпляры таких компонентов будут устанавливаться в проекты пользователя.

Так же здесь необходимо добавить, что назначение задержек актуально только для симуляции проекта. Если разработчику нужна только RTL-модель, то назначение задержек можно не делать. Если же речь идет о модели, полученной после размещения проекта на кристалле, то в такой модели используются задержки, полученные в результате работы компилятора и трассировщика.

```
module modxor (axorb, a, b);
parameter size=8, delay=15;
output [size-1:0] axorb;
input [size-1:0] a, b;
wire [size-1:0] #delay axorb = a ^ b;
endmodule
```

Пример 2.40. Задание задержек при назначениях

5.6 Минимальная, типовая, максимальная задержка (*Minimum, Typical, Maximum Delay*)

Языке Verilog HDL позволяет для каждого из упомянутых выше типов задержки (rise, fall, turn-off) задать минимальное, максимальное и среднее значение. Эти значения разделяют двоеточиями. Синтаксис такого выражения следующий:

```
<mintypmax_expression>
::= <expression>
||= <expression1> : <expression2> : <expression3>
```

При этом необходимо указывать либо все три значения (min, typ, max), либо только одно.

При этом соблюдаются следующие соотношения:

- значение **expression1** меньше или равно значению **expression2**
- значение **expression2** меньше или равно значению **expression3**

Синтаксис для задержек на примитивах типа вентилях (включая примитивы, определяемые пользователем), цепи, и непрерывные назначения позволяет иметь три значения каждое для повышающего фронта, спадающего фронта, и задержки на выключение. В следующем примере 2.42 показано использование задержки со значениям min/typ/max для положительного фронта, отрицательного фронта, и задержки на выключение:

```

module iobuf(io1, io2, dir);
...
...
..

bufif0 #(5:7:9, 8:10:12, 15:18:21) (io1, io2, dir);
bufif1 #(6:8:10, 5:7:9, 13:17:19) (io2, io1, dir);
...
...
...

endmodule

```

Пример 2.42. Пример синтаксиса для задержки.

5.7 Процедурное назначение

Процедурные назначения могут быть использованы для присвоения значений переменным типов `reg`, `integer`, `real` или `time`. Значение присвоенное переменной будет оставаться неизменным, пока не произойдет новое присвоение. Этот механизм существенно отличается от принципа работы непрерывного присваивания, при котором моделируется структура. То есть непрерывное присваивание моделирует постоянную связь между двумя объектами такую, что всякое последующее изменение выражения в правой части присваивания передается в левую часть присваивания. При процедурном же назначении левая часть присваивания изменится только при следующем назначении. Синтаксис непрерывного присваивания:

```

assignment ::= variable_lvalue = [ delay_or_event_control ]
              expression

```

В левой части может стоять одноразрядная или многоразрядная переменная, часть вектора, конкатенация.

Непрерывные присваивания могут выполняться только в рамках последовательных блоков. Последовательный блок объявляется операторами **always** и **initial**. Все операторы, входящие в блок, будут выполняться **последовательно**.

Оператор Initial

Выполнение блок `initial` начинается в нулевой момент модельного времени (при запуске моделирования). Этот блок выполняется только единожды в процессе моделирования. Блоков `initial` может быть несколько, в этом случае они выполняются параллельно и независимо.

Действие оператора `initial` распространяется на один последующий оператор, если нужно распространить его действие на несколько операторов, то используются ключевые слова `begin`, `end` (аналогично `{ }` языка Си). Таким образом, операторы в каждом последовательном блоке выполняются последовательно, но все последовательные блоки, входящие в модуль, выполняются параллельно.

```

module stimulus;

```

```

reg x,y, a,b, m;

initial
    m = 1'b0; //single statement; does not need to be grouped

initial
begin
    #5 a = 1'b1; //multiple statements; need to be grouped
    #25 b = 1'b0;
end

initial
begin
    #10 x = 1'b0;
    #25 y = 1'b1;
end

initial
    #50 $finish;

endmodule

```

В данном примере три оператора `initial` начинают выполняться одновременно в нулевой момент времени. Если перед некоторым оператором имеется выражение задержки «`#...`», то оператор выполняется через время, заданное этой задержкой, относительно предыдущего оператора. Порядок выполнения операторов для данного примера показан в таблице:

время	оператор
0	<code>m = 1'b0;</code>
5	<code>a = 1'b1;</code>
10	<code>x = 1'b0;</code>
30	<code>b = 1'b0;</code>
35	<code>y = 1'b1;</code>
50	<code>\$finish;</code>

Из таблицы видно, что время во всех блоках `initial` протекает независимо, т.е. все они выполняются параллельно.

Оператор `always`

Оператор `always` аналогично оператору `initial` начинает последовательный блок в который также может входить один или несколько операторов. В последнем случае они должны группироваться ключевыми словами `begin ... end`.

Блок `always` также начинает исполняться в нулевой момент времени, но в отличие от `initial` его выполнение циклически повторяется в процессе моделирования. Таким образом оператор `always` хорошо подходит для моделирования регулярно повторяющихся действий в цифровых схемах. Классическим примером применения оператора `always` является построение генератора тактовых сигналов:

```

module clock_gen (output reg clock);

//Initialize clock at time zero
initial
    clock = 1'b0;
//Toggle clock every half-cycle (time period = 20)
always
    #10 clock = ~clock;

initial
    #1000 $finish;
endmodule

```

В данном примере новый цикл выполнения блока **always** начинается сразу же по окончании предыдущего цикла. Можно указать и другую периодичность выполнения блока с помощью событийного выражения (event-control):

```

always @ ( event-expression [or event-expression*] )
begin
... statements ...
end

```

Пример. Синтаксис блока **always**.

Событийное выражение (event) или просто событие соответствует изменению значения в цепи или регистре. Такое событие может использоваться для запуска выполнения последовательного блока связанного с оператором **always**. События могут быть связаны ключевым словом **or**, тогда выполнение блока (т.е. пересчет всех значений, входящих в него) начинается при изменении любой из переменных, входящих в событийное выражение. В таком случае список сигналов в событийном выражении называется списком чувствительности (sensitivity list) оператора **always**. Событийному выражению должен предшествовать символ **@**:

```

@(clock) q = d; //q = d присвоение выполняется при изменении сигнала clock
@(posedge clock) q = d; //q = d присвоение выполняется по каждому поло-
                        //жительному фронту clock ( 0 to 1, x or z,
                        // x to 1, z to 1 )
@(negedge clock) q = d; //q = d то же по отрицательному фронту
                        //( 1 to 0, x or z,
                        //x to 0, z to 0)
q = @(posedge clock) d; //d выражение d вычисляется немедленно, но присва-
                        //ивание q происходит по положительному фронту clock

```

Событийное выражение может иметь имя и выступать в качестве регистровой переменной. Такую переменную можно в дальнейшем использовать в операторе **always**:

```

event received_data; //переменная событийного типа

always @(posedge clock) //по каждому положительному фронту
begin
    if(last_data_packet) //Если последний пакет данных, то
        ->received_data; //срабатывает событие
end

```

```
always @(received_data) //по событию происходит захват данных в буфер
    data_buf = {data_pkt[0], data_pkt[1], data_pkt[2], data_pkt[3]};
```

В зависимости от содержимого событийного выражения на основе блоков **always** могут быть синтезированы как триггеры-защелки или триггеры, срабатывающие по фронту сигнала, так и комбинационная логика. Если в событийном выражении присутствуют ключевые слова **posedge** и/или **negedge**, то будет синтезирована синхронная схема. Если же просто перечисляются сигналы, по изменению которых блок должен исполняться, то будет синтезирована комбинационная логика.

```
// синхронная схема
always @ ( posedge clock ) begin
... statements ...
end

//комбинационная логика
always @(a or b or c) begin
    f = a & b & c;
end
```

Совмещение в одном списке чувствительности событий связанных с фронтом сигнала и с уровнем не допустимо.

```
// неправильная запись
always @ ( posedge clock or c) begin
... statements ...
end
```

Блок **always** может также содержать логику, выходная информация которой защелкивается на триггерах при изменении уровня сигнала или на переднем или заднем фронте сигнала:

```
always @(posedge clock) begin
    f = a & b & c;
end
```

Любому блоку в модуле на языке Verilog может быть присвоено имя (named blocks). Имя указывается после ключевого слова **begin** и отделяется от него двоеточием (см. пример ниже). В именованных блоках можно объявлять локальные переменные и получать к ним доступ через иерархическое имя.

```
module top;

initial
begin: block1 //последовательный блок с именем block1
integer i; //целое i – статическая локальная переменная в блоке block1
           // к ней можно обратиться по иерархическому имени, top.block1.i
           //(например из блока более высокого уровня)
...
...
end
```

```
end
```

Исполнение именованных блоков можно прерывать, об этом будет сказано подробнее в следующей части пособия.

Блокирующие и неблокирующие присваивания

Блокирующее присваивание запрещает исполнение других присваиваний до своего завершения. Это гарантирует строго последовательное исполнение операторов в блоке, даже если в записи результата (в правой части оператора присваивания) присутствует опция задержки. Если присвоение содержит опцию задержки, то изменение будет выполнено через то число квантов модельного времени, которое соответствует значению задержки. При этом процесс моделирования как бы приостанавливается, что выражается в том, что следующий оператор в данном блоке будет выполнен только после выполнения данного оператора. Сразу после выполнения блокирующего присваивания его результат становится непосредственно доступным любому последующему оператору в текущем программном блоке. Если имеется несколько последовательных операторов со своими указаниями времени, то время задержки очередного присвоения от начала исполнения первого есть сумма задержек предшественников.

Интересно, что если в блоке ни один оператор не содержит признаков приостановки исполнения, то с точки зрения окружения все операторы в блоке выполняются как бы одновременно, и никакие промежуточные преобразования "невидимы" для других блоков.

Неблокирующее присваивание отображается сочетанием символом `<=` и разрешает исполнение последующих операторов еще до собственного завершения. То есть система моделирования как бы не останавливается до выполнения оператора присваивания, а назначает присваивание на соответствующий момент времени и продолжает обрабатывать последующие операторы.

Более того, если совокупность неблокирующих операторов инициирована общим событием, то эти операторы начинают исполняться одновременно с точки зрения модельного времени, т. е. все они используют значения операндов, определенные до начала исполнения всей совокупности. Новые значения запоминаются в буфере до исполнения всех операторов, вызванных общим событием. Из этого видно важное различие между блокирующими и неблокирующими присваиваниями: результат первого доступен сразу после его выполнения и учитывается всеми последующими операторами, а результат неблокирующего присваивания будет «виден» только после выполнения всего блока.

```
//non_block1.v
module non_block1(out,);
//input
output out;
reg a, b, c, d, e, f;
//blocking assignments
initial
begin
a = #10 1;
b = #2 0;
c = #4 1;
```

Симулятор назначает значение 1 в регистр **reg a** в момент времени симуляции – 10, назначает значение 0 в регистр **reg b** в момент времени симуляции – 12, и назначает значение 1 регистр **reg c** в момент времени

```

end
//non-blocking assignments
initial
begin
d <= #10 1;
e <= #2 0;
f <= #4 1;
end
initial
begin
$monitor ($time, , "a = %b
  b = %b c = %b d = %b e =
  %b
  f = %b", a,b,c, d,e,f);
  #100 $finish;
end
endmodule

```

симуляции - 16.

Симулятор назначает значение 1 в регистр **reg d** в момент времени симуляции 10, назначает значение 0 в регистр **reg e** в момент времени симуляции 2, и назначает значение 1 в регистр **reg f** в момент времени симуляции 4.

Описывать цепочку регистров необходимо с использованием неблокирующего процедурного назначения:

```

always @ (posedge CLK)
begin
    r1 <= a;
    r2 <= r1;
    r3 <= r2;
end

```

Каждый элемент в этой цепочке дает задержку на один такт синхросигнала.

Объединять в одном последовательном блоке блокирующие и неблокирующие присваивания не рекомендуется.

6 Операторы ветвления

6.1 Утверждения *if...else*

Утверждения **if...else** позволяют выполнять блок утверждений в зависимости от результата проверки на истинность или ложность выражений, определяющих логику выбора.

В соответствии со значением этих выражений и производится обработка блока утверждений, входящих в конструкцию **if...else**

Синтаксис утверждения **if...else** показан в примере 2.82.

```

if ( expr )
begin
... statements ...
end
else

```

```
begin
... statements ...
end
```

Пример 2.82. Синтаксис утверждения **if...else**.

Условный оператор **if** состоит из ключевого слова **if** и сопровождается выражением в круглых скобках. Условный оператор **if** сопровождается утверждением, или блоком утверждений, включенных от слова **begin** и заканчивающихся словом **end**. Если значение выражения является отличным от нуля, то выражение истинно (**true**) и блок утверждений, который следует в этой конструкции, выполняется. Если значение выражения - нуль, то выражение – ложно (**false**) и блок утверждений, который следует в этой конструкции, не выполняется.

Дополнительное утверждение **else** может следовать за оператором **if**. Если выражение следующее за **if**, ложно, то выполняется утверждение или блок утверждений следующей за **else**.

Утверждения **if...else** могут привести к синтезу регистра. Регистры синтезируются в том случае, когда Вы не назначаете значение этого же регистра **reg** во всех ветвлениях условной конструкции. Компилятор HDL синтезирует логику мультиплексора (или подобную логику выбора) от одного оператора **if**. Условное выражение в операторе **if** синтезируется как сигнал управления на мультиплексор, который определяет выбор сигналов, проходящих через мультиплексор. Например, утверждения в примере 2.83 создают логику мультиплексора, которой управляет входом **c** и переключает входы **a** или **b** на выход переменной **x**.

```
if (c)
x = a;
else
x = b;
```

Пример 2.83. Оператор **if**, который синтезируется как мультиплексор.

В примере 2.84 показано как можно использовать выражение **if...else**, чтобы создать структуру произвольной длины.

```
if (instruction == ADD)
begin
carry_in = 0;
complement_arg = 0;
end
else if (instruction == SUB)
begin
carry_in = 1;
complement_arg = 1;
end
else
illegal_instruction = 1;
```


Можно использовать вложенные выражения **if...else**:

```
if (select[1])
begin
if (select[0]) out = in[3];
else out = in[2];
end
else
begin
if (select[0]) out = in[1];
else out = in[0];
end
```

В примерах приведенных выше, за выражением **if** всегда следовало выражение **else**. Однако, если не делать выражения **else** и не определять состояние сигналов по **else**, то такая переменная будет иметь только одно назначение по условию **if** и для такой переменной компилятор HDL может синтезировать триггер-защелку. Переменная будет назначена по условию в том случае, если есть хотя бы один путь, который явно не назначает значение на эту переменную. В примере 2.86 показана, переменная **value**, управляется по условию. Если **c** не истинно, то переменная **value** не имеет назначения и сохраняет свое предыдущее значение.

```
always begin
if ( c )
begin
value = x;
end
Y = value; // causes a latch to be
           // synthesized for value
end
```

Пример 2.86. В данном примере для переменной, управляемой по условию, синтезируется триггер-защелка.

Следует помнить, что конструкция **if...else if...else** дает приоритетную схему: если первое условие не выполняется, то проверяется второе, если второе не выполняется, то проверяется третье и так далее, до первого истинного условия. После обнаружения условия, результат которого истина, выполняется соответствующая ему ветвь оператора **if**, прочие условия **else if** (в том же операторе **if**) уже не проверяются! Если нужно, чтобы все условия проверялись одновременно, то необходимо записать отдельный оператор **if** для каждого из них. При этом следует избегать конфликтов по присваиванию в разных операторах **if**, т.е. не должно возникать ситуаций, когда одной и той же переменной одновременно присваиваются значения в разных операторах **if**.

Приоритетная схема требует больше ресурсов для реализации, чем не приоритетная, поэтому следует, по возможности, стремиться к использованию отдельных операторов **if** вместо конструкции **if...else if...else if ...else if ...else**. Однако на практике, все же, чаще приходится

пользоваться приоритетной схемой (по убыванию приоритета):

- асинхронный сброс (установка)
- синхронный сброс (установка)
- прочие управляющие сигналы

6.2 Утверждения **case**

Функции, выполняемые утверждением **case** подобны тем, что выполняются в утверждениях **if...else**. Утверждение **case** позволяет выполнить многократные ветвления цепей в логике, которые выполняются на значениях выражения. Один способ описывать схему с многократными ветвлениями – с использованием утверждения **case** (см. пример). Другой способ - с несколькими утверждениями **@** (clock edge), которые обсуждаются в последующих разделах, описывающих применение циклов.

Синтаксис для утверждения **case** показан в примере 2.87.

```
case ( expr )
case_item1: begin
... statements ...
end
case_item2: begin
... statements ...
end
default: begin
... statements ...
end
endcase
```

Утверждение **case** состоит из ключевого слова **case**, сопровождаемого выражением в круглых скобках, кроме того в конструкцию входят одно или более значений для **case** (и утверждения, с ним связанные, которые будут выполняться), сопровождаемые ключевым словом **endcase**. Значения для **case** состоят из выражения (обычно это простые константы) или списка выражений, отделенных запятыми, сопровождаемыми двоеточием (:).

При выполнении выражения **case**, производится сравнение значения выражения, следующего после ключевого слова **case**, по-очередно с каждым из вариантов значений, находящихся в конструкции **case**. Когда выражение равно значению в одном из вариантов в конструкции, то это условие оценивается как истина - **true**. В каждом из вариантов значений **case** могут быть записаны несколько выражений, отделенных друг от друга запятыми. Когда используются несколько выражений, то условие будет истинным в том случае, если любое из выражений в теме **case** соответствует выражению после ключевого слова **case**. Первое же значение, которое будет оцениваться как истина в **case**, определит выбор пути. Все последующие значения для **case** игнорируются, даже если они истинны. Если ни одно из значений, находящихся в **case** не выбрано, то никаких действий не будет выполнено.

Вы можете определить выбор значения в **case** по умолчанию - **default** и соответственно приписать ему выражения умолчанию, которые будут использоваться, когда не выбрано никакое другое значение в **case**. Пример утверждения **case**:

```
case (state)
```

```

IDLE: begin
if (start)
next_state = STEP1;
else
next_state = IDLE;
end
STEP1: begin
//do first state processing here
next_state = STEP2;
end
STEP2: begin
//do second state processing here
next_state = IDLE;
default: next_state = IDLE;
end
endcase

```

6.3 Оператор *casex*, *casez*

Версии оператора *casex* и *casez* позволяют не учитывать при проверке совпадения состояние некоторых разрядов. *casez* не учитывает значение разрядов, имеющих значение *z* как в записи вариантов, так и в результате вычисления ключевого выражения. Так коды 7'b00zz101 и 7'bzz1010z с точки зрения оператора *casez* эквивалентны.

Оператор *casex* не учитывает при сравнении разряды установленные в состояние высокого импеданса или в неопределенное состояние. Так же допускается использование символа ? вместо символов *x* и *z*.

Ниже показан сложный фрагмент кода, не использующий *casex*

```

if (cond[3]) out = 0;
else if (!cond[3] & cond[2] ) out = 1;
else if (!cond[3] & !cond[2] & cond[1] ) out = 2;
else if (!cond[3] & !cond[2] & !cond[1] & cond[0] ) out = 3;
else if (!cond[3] & !cond[2] & !cond[1] & !cond[0] ) out = 4;

```

В примере 2.95 показан тот же фрагмент кода, но значительно упрощенный, благодаря использованию **casex** с символом ?

```

casex (cond)
4'b1???: out = 0;
4'b01??: out = 1;
4'b001?: out = 2;
4'b0001: out = 3;
4'b0000: out = 4;

```

```
endcase
```

Пример использования оператора **casez** с использованием символов z Показан ниже

```
casez (what_is_it)
2'bz0: begin
    //accept anything with least significant bit zero
    it_is = even;
end
2'bz1: begin
    //accept anything with least significant bit one
    it_is = odd;
end
endcase
```

7 Иерархическое описание проекта

Большинство современных языков проектирования аппаратуры поддерживают возможность описания проекта в виде совокупности заранее описанных компонентов и их связей, и язык Verilog не является исключением. Всякая иерархия представлена главным проектным модулем, который называют вершиной проекта, и совокупности подчиненных проектных модулей. Вершина проекта содержит операторы вхождения компонентов. Подчиненный проектный модуль может быть, в свою очередь, вершиной следующей иерархии. В общем случае, проектные модули независимы в том смысле, что каждый может использоваться самостоятельно в различных конструкциях и быть вершиной проекта. Проектные модули, относящиеся к одному проекту, Могут находиться в одном файле или представляться несколькими файлами. В последнем случае проектные файлы должны компилироваться в библиотеку проекта в порядке их вхождения в иерархию снизу вверх.

Вхождение проектного модуля в проект высшего иерархического уровня отображается оператором вхождения, формальный синтаксис которого определяется следующим образом:

<оператор вхождения> :: =

<имя модуля> [<присвоение значений параметрам>] <объявление экземпляра модуля>
«,<объявление экземпляра модуля> »;

<присвоение значений параметрам> :: =

(<выражение> «,<выражение> »)

<объявление экземпляра модуля> :: =

<имя экземпляра> (список соединений)

<список соединений> :: =

<соединение порта модуля> «,<соединение порта модуля> »;

| <именованное соединение> «,<именованное соединение> »

<соединение порта модуля> :: = <выражение> | <пробел>

<именованное соединение> :: = . <имя порта> ([<выражение>])

При описании вхождения модуля в иерархический проект используются два имени: имя вхождения и имя модуля. Первое присваивает собственное имя включаемой подсхеме, а второе определяет способ ее функционирования и реализации в форме указания имени программного модуля, описывающего его функционирование или структуру. Несколько вхождений могут быть представлены однотипными подсхемами, т. е. ссылаться на одинаковый программный модуль, но каждый экземпляр имеет собственное имя. Как видно из синтаксических правил, допустимо объявление нескольких однотипных встраиваемых модулей в одной декларации.

Оператор вхождения может рассматриваться как вызов подпрограммы с проблемно-ориентированным интерфейсом вызова. Кроме того, отличие состоит в том, что внутренние переменные включаемого модуля определены как статические, сохраняющие значения между инициализациями. Формальные объекты включаемого модуля (параметры и сигналы) при инициализации оператора замещаются фактическими объектами вызывающей программы. Оператор вхождения модуля обязательно параллельный, это означает, что преобразования, определенные в операторной части включаемого модуля, выполняются всякий раз, когда изменяются фактические объекты оператора вхождения.

Сопоставление формальных и фактических переменных при вызове осуществляется либо в соответствии с позицией в соответствующем списке, либо по имени (именованное соединение). Разные способы сопоставления в одном операторе недопустимы. При позиционном сопоставлении порядок записи фактических объектов в списке соединений порта точно соответствует порядку записи имен портов в тексте описания модуля. Неиспользуемые порты, а также входные порты, для которых значение сигналов выбирается заданным по умолчанию, отмечаются символом <пробел>. При сопоставлении по имени описание подключения порта начинается со знака "точка", после чего записано имя порта, а затем, в скобках, имя сигнала или выражение, вычисляющее значение сигнала, подаваемого на порт.

Пример подключения модуля:

```
module ripple_carry_counter(q, clk, reset);

output [2:0] q;
input clk, reset;

// Создается три экземпляра модуля T_FF, каждый из них имеет уникальное имя и
// свой набор подключенных сигналов.

// Вариант 1 - позиционное сопоставление портов
T_FF tff0(q[0],clk, reset);
T_FF tff1(q[1],q[0], reset);
T_FF tff2(q[2],q[1], reset);

// Вариант 2 - именное сопоставление портов
T_FF tff0(.q(q[0]),
          .clk(clk),
          .reset(reset));
T_FF tff1(.q(q[1]),
          .clk(q[0]),
```

```

        .reset( reset));
T_FF tff2(.q(q[2]),
        .clk(q[1]),
        .reset( reset));
endmodule

```

В данном примере показан структурный подход к описанию асинхронного счетчика. Базовыми элементами для его построения являются Т-триггеры, описанные ранее как:

```

module T_FF(q, clk, reset);

output q;
input clk, reset;
wire d;

D_FF dff0(q, d, clk, reset);
assign d =~q;

endmodule

```

Если подключаемый модуль имеет параметры, то их значения могут быть переопределены при подключении модуля. Сопоставление формальных и фактических параметров также может быть позиционным или именовым. В первом случае список присваиваемых при вызове фактических значений параметров предшествует имени вхождения. Порядок записи фактических значений параметров строго соответствует порядку объявления формальных имен параметров в описании включаемого модуля.

При именовом сопоставлении используется ключевое слово `defparam`. Синтаксис его использования приведен ниже:

```
defparam <имя экземпляра модуля>.<имя параметра> = <значение параметра> {, <имя параметра> = <значение параметра>;}
```

Пример:

```

//объявляем модуль с параметрами
module bus_master;
parameter delay1 = 2;
parameter delay2 = 3;
parameter delay3 = 7;
...
<module internals>
...
endmodule

//модуль верхнего уровня; instantiates two bus_master modules
module top;

//Instantiate the modules with new delay values

// Позиционное сопоставление
bus_master #(4, 5, 6) b1(); //b1: delay1 = 4, delay2 = 5, delay3 = 6
bus_master #(9, 4) b2(); //b2: delay1 = 9, delay2 = 4, delay3 = 7(default)

//именное сопоставление

```

```

bus_master b3();
    defparam b3.delay2 = 4,           //b2: delay2 = 4, delay3 = 7
           delay3 = 7;               //delay1=2 (default)

endmodule

```

Как для сопоставления портов, так и параметров рекомендуется использовать именное, а не позиционное сопоставление. Это позволяет избежать ошибок возможных при изменении состава и порядка следования портов/параметров подключаемого модуля.

7.1 Подключение массива модулей

При необходимости подключения к проекту нескольких идентичных экземпляров одного модуля можно использовать описание подключения в виде массива, например:

```

nand #2 t_nand[0:7] ( ... );
nand #2 x_nand[0:3] ( ... ), y_nand[4:7] ( ... );

```

В данном примере объявляются 16 экземпляров модуля `nand`. Доступ к экземпляру модуля, в случае необходимости, осуществляется по индексу.

При подключении портов модулей действуют следующие правила:

- если разрядность подключаемой цепи совпадает с разрядностью порта одного экземпляра модуля, то цепь подключается ко всем модулям массива параллельно;
- если разрядность подключаемой цепи отличается от разрядности порта одного экземпляра модуля, то к порту каждого модуля в массиве подключается соответствующая часть цепи (диапазон разрядов);
- если разрядность подключаемой цепи слишком мала или велика для подключения к портам всех модулей в массиве, это считается ошибкой.

Правила проиллюстрируем примерами. Первый пример соответствует второму положению правил. Все порты подключаемого модуля `bufif0` одноразрядные. Модули `driver` и `driver_equiv` в приведенном примере эквивалентны.

```

module driver (in, out, en);
input [3:0] in;
output [3:0] out;
input en;
bufif0 ar[3:0] (out, in, en); // array of three-state buffers
endmodule

module driver_equiv (in, out, en);
input [3:0] in;
output [3:0] out;
input en;
bufif0 ar3 (out[3], in[3], en); // each buffer declared separately
bufif0 ar2 (out[2], in[2], en);
bufif0 ar1 (out[1], in[1], en);
bufif0 ar0 (out[0], in[0], en);
endmodule

```

Следующий пример иллюстрирует вариант подключения массива модулей с

векторными портами. Аналогично предыдущему примеру `busdriver` и `busdriver_equiv` эквивалентны.

```
module busdriver (busin, bushigh, buslow, enh, enl);  
input [15:0] in;  
output [7:0] bushigh, buslow;  
input enh, enl;  
driver busar3 (busin[15:12], bushigh[7:4], enh);  
driver busar2 (busin[11:8], bushigh[3:0], enh);  
driver busar1 (busin[7:4], buslow[7:4], enl);  
driver busar0 (busin[3:0], buslow[3:0], enl);  
endmodule  
  
module busdriver_equiv (busin, bushigh, buslow, enh, enl);  
input [15:0] busin;  
output [7:0] bushigh, buslow;  
input enh, enl;  
driver busar[3:0] (.out({bushigh, buslow}), .in(busin),  
.en({enh, enh, enl, enl}));
```